SAT vs. CP-SAT vs. Classical Backtracking for Generalized Sudoku: An Empirical Benchmark

Pham Gia Bach - 24021382[a,*]

ARTICLE INFO

Keywords:
Sudoku
SAT encoding
CP-SAT
classical backtracking
constraint programming
benchmarking
AMO encoding

ABSTRACT

This paper presents an empirical comparison of three solver families—SAT, constraint programming (CP-SAT), and classical backtracking—on a shared benchmark of generalized Sudoku instances. We evaluate seven PySAT configurations spanning five At-Most-One (AMO) encodings (pairwise, sequential counter, binary, commander, product) across three CDCL backends (CaDiCaL 1.5.3, Glucose 4, MiniSat 2.2), Google OR-Tools CP-SAT, and an optimized backtracking implementation using minimum-remaining-values (MRV) cell selection with forward checking. The benchmark comprises 120 Sudoku instances across grid sizes $N \in \{9, 36, 81\}$ and clue-removal ratios $r \in \{0.5, 0.9\}$, with 20 instances per stratum, generated from a single seed (42). Every solver run is subject to a 10.0-second wall-clock timeout, a 60% available-memory resource guard, and independent solution verification. Across 1,000 attempted runs (760 SAT, 120 CP-SAT, 120 backtracking), SAT solvers produce 431 valid solutions (56.71%), CP-SAT produces 64 (53.33%), and backtracking produces 40 (33.33%), all independently verified with zero invalid solutions. Both SAT and CP-SAT achieve 100% solve rates on 9×9 instances and zero valid solutions on 81×81 within the timeout. On 36×36 at $r = 0.5$, solve rates fall below 21% for all families. We find no evidence of universal superiority: pooling seven SAT configurations gives SAT a different aggregation structure than the two single-configuration baselines, and pairwise/binary SAT encodings are predeclared excluded at $N = 81$ by the memory guard, leaving their performance at that size unknown. No confidence intervals, hypothesis tests, or replications are reported; all findings are descriptive and timeout-censored.

1. Introduction

Sudoku is a number-placement constraint satisfaction problem in which an $N \times N$ grid (with $N = k^2$ for box size k) must be completed so that every row, column, and $k \times k$ box contains each digit $1, \ldots, N$ exactly once, consistent with pre-filled clue cells. The decision version of Sudoku is NP-complete (Yato, 2003), which makes it a natural benchmark for comparing combinatorial search paradigms. Despite its simple rules, Sudoku instances exhibit wide variation in solving difficulty depending on grid size and clue density, and no single solving approach dominates across all regimes.

Three solving families are widely used in practice. Boolean satisfiability (SAT) solvers encode Sudoku as a conjunctive normal form formula using one-hot Boolean variables $x[r, c, v]$ and exactly-one (at-least-one plus at-most-one) constraints per cell, row, column, and box. The choice of At-Most-One (AMO) encoding—pairwise, sequential counter, binary, commander, or product—materially affects formula size and propagation strength, and modern CDCL backends differ in restart, conflict-analysis, and clause-deletion heuristics. Constraint programming solvers such as OR-Tools CP-SAT model Sudoku directly with integer variables in domain $1..N$ and global all-different constraints, combining constraint propagation with CDCL-style clause learning. Classical backtracking explores partial assignments with variable and value ordering heuristics, optionally augmented with forward checking or arc consistency.

Prior work has studied SAT encodings for Sudoku (Lynce and Ouaknine, 2006), AMO encoding trade-offs (Sinz, 2005; Nguyen, 2020; Hn, 2013; Klieber and Kwon, 2007; Chen, 2010; Frisch and Giannaros, 2010), and constraint programming for Sudoku (Simonis, 2005; aaa, 2007). However, we are not aware of prior work that directly compares five AMO encodings across three CDCL backends, CP-SAT, and classical backtracking on identical generalized Sudoku instances spanning $N \in \{9, 36, 81\}$, with independent solution validation on every returned candidate.

This study addresses three experimental questions:

*Corresponding author
ORCID(s):

1. **RQ1:** How do five SAT AMO encodings and three CDCL backends compare in valid solve rate and solve time on identical Sudoku instances?
2. **RQ2:** How do SAT-based solvers compare with OR-Tools CP-SAT and classical backtracking in valid solve rate and solve time?
3. **RQ3:** How do grid size ($N \in \{9, 36, 81\}$) and clue-removal ratio ($r \in \{0.5, 0.9\}$) relate to success rate and solve time across solver families?

The defensible contributions of this paper are: (i) a documented benchmark of 1,000 attempted solver runs across three solver families and six problem strata, with raw results and an independent validator; (ii) descriptive evidence that SAT and CP-SAT achieve comparable valid solve rates (56.71% vs. 53.33%) on shared instances, while classical backtracking solves 40 of 120 instances, all confined to $N = 9$; (iii) documentation of a hardness cliff at $N = 36, r = 0.5$ (solve rates below 21% for all families) and intractability at $N = 81$ within a 10-second timeout; and (iv) explicit accounting of predeclared memory-guard exclusions, live resource-guard rejections, and timeout censoring, which limit the inferential strength of all comparisons. We make no claim of universal superiority for any solver family, no claim of statistical significance, and no generalization beyond the tested dataset.

2. Related Work

2.1. SAT formulations and At-Most-One encodings

Lynce and Ouaknine (2006) introduced the first SAT encodings for Sudoku, proposing minimal and extended CNF formulations using one-hot Boolean variables with exactly-one constraints per cell, row, column, and box. They report that unit propagation on the extended encoding can solve approximately half of hard 9×9 puzzles without search. This work established the one-hot Boolean formulation—N^3 primary variables and $4N^2$ exactly-one groups—that underlies all SAT configurations evaluated here. Kwon (2016) presented an optimized CNF encoding for Sudoku aimed at reducing clause count for larger puzzles; it is cited for context only, and no technical claim in this paper depends on it.

Five AMO encodings are implemented and evaluated. Sinz (2005) introduced the sequential counter encoding for Boolean cardinality constraints, using $O(n)$ auxiliary variables and $O(n)$ clauses for an at-most-one constraint over n literals; this is the basis of the sequential encoding in `sat.py`. Klieber and Kwon (2007) proposed the commander encoding, which partitions literals into subgroups with commander variables and a group-size parameter b; the implementation uses $b = 4$. Chen (2010) introduced the product encoding, arranging literals in a two-dimensional grid with row and column indicator variables. Frisch and Giannaros (2010) presented binary/bitwise at-most-one encodings using $\lceil \log_2 n \rceil$ auxiliary variables.

Nguyen (2020) provided an empirical comparison of AMO encodings (pairwise, sequential, binary, commander, product, bimander) across three SAT solvers on generic benchmarks, finding that encoding choice has a measurable impact on solver performance. Our study applies the same five encodings to Sudoku specifically, across multiple grid sizes and three CDCL backends, with independent solution validation. Hn (2013) is a technical survey of AMO encodings with clause and variable count comparisons; it provides the complexity comparison used in Section 3 but reports no empirical Sudoku runtime results.

2.2. Constraint programming, CP-SAT, and classical backtracking

Simonis (2005) modeled Sudoku as a constraint satisfaction problem using all-different constraints on rows, columns, and boxes, contrasting 27 arity-9 all-different constraints with 810 binary inequality constraints. aaa (2007) presented an interactive constraint-based Sudoku solver and confirmed that Sudoku is NP-complete via Yato (2003); this work is educational/interactive in focus and reports no cross-solver benchmarking. OR-Tools CP-SAT (Google, 2026) is a hybrid constraint-programming solver built on a CDCL SAT engine and enriched with MIP techniques; it accepts integer decision variables and global all-different constraints directly. The CP-SAT configuration evaluated here uses N^2 integer variables in domain $1..N$ with all-different constraints per row, column, and box, plus unit equality constraints for clues.

Classical backtracking for Sudoku selects an unassigned variable, tries candidate values, and backtracks on contradiction. The implementation evaluated here uses minimum-remaining-values (MRV) cell selection, forward checking after each assignment, and optional AC-3 propagation over binary peer inequalities

(disabled in the benchmark runs). Candidates are tried in ascending order; MRV ties break on lowest cell index (row-major). No least-constraining-value (LCV) heuristic and no degree heuristic are implemented; see Section 3.4.

2.3. Solver benchmarking and reproducibility

Nguyen (2020) and Sheng et al. (2025) discuss encoding-size versus runtime trade-offs and the importance of empirical evaluation over theoretical clause counts alone. This study contributes an empirical Sudoku-specific comparison but shares the limitations of single-seed, single-run benchmarks: no confidence intervals, no replication, and timeout-censored runtimes preclude inferential claims (Sections 5–7).

3. Problem Formulation and Solver Methods

3.1. Generalized $N \times N$ Sudoku

A Sudoku puzzle of size $N \times N$, where $N = k^2$ for box size k, consists of an $N \times N$ grid with some cells pre-filled (clues) and the remainder empty. The objective is to fill every empty cell with a digit from 1 to N such that:

1. each row contains each digit exactly once;
2. each column contains each digit exactly once;
3. each $k \times k$ box contains each digit exactly once;
4. every pre-filled clue matches the corresponding cell in the solution.

A solution is valid if it satisfies all constraints. Solution uniqueness is not required: any valid completion matching the clues counts as correct, as implemented in `is_valid_solution_for_puzzle` (`src/sudoku/validator.py`).

3.2. SAT formulation and AMO encodings

The SAT model uses N^3 one-hot Boolean variables $x[r, c, v]$, where $x[r, c, v] = 1$ iff cell (r, c) contains digit v, mapped to 1-based index $r \cdot N^2 + c \cdot N + v$. The constraints are:

1. **Cell exactly-one**: each cell contains exactly one digit (N^2 groups of size N).
2. **Row exactly-one**: each digit occurs once per row (N^2 groups).
3. **Column exactly-one**: each digit occurs once per column (N^2 groups).
4. **Box exactly-one**: each digit occurs once per box (N^2 groups).
5. **Clues**: positive unit clauses for pre-filled cells.

There are $4N^2$ exactly-one groups in total. Each group is encoded as one at-least-one (ALO) clause plus one at-most-one (AMO) encoding chosen from five alternatives implemented in `src/sudoku/solvers/sat.py`:

- **Pairwise**: forbids every pair of literals in the group; 0 auxiliary variables, $N(N-1)/2$ clauses per group, $O(N^2)$ clauses per group.
- **Sequential counter** (2005): prefix variables record whether a literal in the prefix is true; $N - 1$ auxiliary variables, $O(N)$ clauses per group.
- **Binary** (2010): each literal forces a distinct binary bit pattern on auxiliary variables; $\lceil \log_2 N \rceil$ auxiliary variables, $O(N \log N)$ clauses per group.
- **Commander** (2007): literals are partitioned into subgroups of size b (default $b = 4$) with commander variables; $\lceil N/b \rceil$ auxiliary variables per group.
- **Product** (2010): literals are arranged in a 2D grid with row and column indicator variables; approximately $\lceil \sqrt{N} \rceil + \lceil N/\lceil \sqrt{N} \rceil \rceil$ auxiliary variables per group.

Clue cells are encoded as positive unit clauses. PySAT CDCL backends evaluated are CaDiCaL 1.5.3, Glucose 4, and MiniSat 2.2, each invoked through the PySAT `Solver` interface with bootstrapped clauses. PySAT's `solve()` does not accept a wall-clock timeout directly; the 10.0-second limit is enforced externally by the subprocess runner via SIGKILL (`src/sudoku/bench/runner.py`).

Table 1
Solver configurations in the primary benchmark.

Label	Family	Encoding	Backend	Threads
sat_pairwise_cadical153	SAT	pairwise	cadical153	1
sat_sequential_cadical153	SAT	sequential	cadical153	1
sat_binary_cadical153	SAT	binary	cadical153	1
sat_commander_cadical153	SAT	commander ($b = 4$)	cadical153	1
sat_product_cadical153	SAT	product	cadical153	1
sat_sequential_glucose4	SAT	sequential	glucose4	1
sat_sequential_minisat22	SAT	sequential	minisat22	1
cpsat	CP-SAT	—	ortools-cpsat	1

3.3. CP-SAT formulation

The CP-SAT model uses N^2 integer decision variables $x[r, c] \in \{1, \ldots, N\}$ with all-different constraints per row, column, and box, plus unit equality constraints for clues. OR-Tools CP-SAT combines constraint propagation with CDCL-style conflict analysis. The solver parameter `max_time_in_seconds` is set to the 10.0-second timeout, and `num_workers` is set to 1 (`src/sudoku/solvers/cpsat.py`).

3.4. Classical backtracking

The backtracking solver (`src/sudoku_solver/backtracking.py`) maintains a domain of candidate values for each cell, initialized to $\{1, \ldots, N\}$ and narrowed by clues. It implements:

- **MRV cell selection**: the unassigned cell with the fewest remaining candidates is chosen; ties break on lowest cell index (row-major); a cell with an empty domain is returned immediately as a contradiction.
- **Forward checking**: after assigning a value, that value is removed from the domains of all peer cells (sharing a row, column, or box); an emptied domain fails the branch.
- **Optional AC-3**: arc consistency over binary peer inequalities is available via `use_ac3`, but is disabled in the benchmark runs (`use_ac3=False`); AC-3 on binary inequalities is weaker than generalized arc consistency on all-different constraints.
- **Depth-first search**: candidates are tried in ascending order; each decision narrows the decision cell's domain to a singleton, propagates, recurses, and restores on failure.
- **Timeout**: the search aborts once a wall-clock deadline is reached and returns timeout status.

Source inspection confirms that **no least-constraining-value (LCV) heuristic and no degree heuristic are implemented**; candidates are sorted ascending and MRV ties break deterministically on cell index. The benchmark invokes the solver with `use_ac3=False` and `randomize=False` (`src/sudoku/bench/runner.py:run_solver_in_process`).

3.5. Solver configurations

Eight configurations were in the primary benchmark matrix: seven SAT configurations and one CP-SAT configuration. Table 1 lists them. Gurobi and CPLEX adapters exist in the codebase but were not executed due to commercial license restrictions and are excluded from this study. The backtracking solver is evaluated separately; its results are taken from `data/results/backtracking_results.csv` (Section 3.4).

4. Experimental Design

Figure 1 summarizes the experimental workflow.

4.1. Dataset

The benchmark uses 120 Sudoku instances spanning six difficulty strata: grid sizes $N \in \{9, 36, 81\}$ (box sizes $k \in \{3, 6, 9\}$) and clue-removal ratios $r \in \{0.5, 0.9\}$, with 20 instances per stratum. These details are confirmed by `data/dataset_info.json` (base seed 42, generator version gen-v1, 20 puzzles per pair, 6 pairs, 120 total puzzles) and `data/manifest.csv`.

Puzzle generation (`src/sudoku/generator.py`, gen-v1): construct a valid complete grid from the base pattern $(k \cdot (r \bmod k) + r \div k + c) \bmod N + 1$; apply seeded validity-preserving transformations (row permutations within bands, column permutations within stacks, band/stack permutation, optional transpose, digit relabeling); validate the complete grid; remove $n_{\text{remove}} = \text{round_half_up}(r \times N^2)$ cells; validate the puzzle against the reference solution with `is_valid_puzzle`. Puzzles are not guaranteed to have unique solutions; any valid completion matching clues counts as correct.

The realized removal ratios deviate slightly from requested values due to `round_half_up` and cell-count quantization (e.g., $N = 9, r = 0.5$ realizes as 0.50617 with 40 clues; $N = 81, r = 0.9$ realizes as 0.90002 with 656 clues). All puzzles are loaded from stored files; no puzzle regeneration occurs during benchmarking.

Generator limitations (from source documentation): puzzles are sampled from the automorphism equivalence orbit of the standard base pattern, not uniformly from all valid Sudoku grids; removal ratio is an experimental condition, not a difficulty label; only two ratios are tested.

4.2. Solver configurations, timeout, and resource guard

Each run executes in an isolated subprocess (`subprocess.Popen` with `start_new_session=True`) under a 10.0-second wall-clock timeout enforced by SIGKILL via the process monitor. A resource guard estimates CNF build memory for SAT encodings: estimated_bytes = clauses × bytes_per_clause; if this exceeds 0.60 × available RAM, the run is recorded as `BUILD_FAIL` with `stats.predeclared_exclusion=true` and is *not* counted as an attempted run. The guard affects $N = 81$ SAT pairwise and binary encodings only (80 predeclared exclusions total: 40 pairwise + 40 binary across both $N = 81$ strata). One live resource-guard rejection occurred during execution (commander at $N = 81, r = 0.5$); this is counted as an attempted run but not as a valid solution.

Every returned solution is independently verified by `is_valid_solution_for_puzzle` (`src/sudoku/validator.py`), which checks that the candidate is a complete legal grid and matches every clue. A run counts as `SOLVED_VALID` only if the solution passes validation.

4.3. Measurement protocol

Runtime is measured as wall-clock elapsed time using `time.perf_counter()` in the parent benchmark process, encompassing subprocess startup, model building, solving, result extraction, and independent solution validation. Three timing fields are recorded: `t_build` (model construction), `t_solve` (pure solver solve time), and `t_total` (end-to-end). For timed-out runs, `t_total` records the wall-clock elapsed time until SIGKILL is delivered to the worker's process group; `t_build` and `t_solve` are not recorded.

Peak memory usage is measured as peak resident set size (RSS), sampled every 20 ms via `psutil.Process.memory_info()` and cross-checked against the child process's `ru_maxrss`. The reported `peak_rss_mb` is the maximum of these two measurements, minus the worker's baseline RSS measured immediately before the solver invocation. Memory is reported in mebibytes (MiB, 1 MiB = 1024^2 bytes).

The memory resource guard is a pre-execution estimate only: it compares the estimated CNF build memory (clause count × bytes-per-clause, calibrated from pilot runs) against 60% of available RAM before the solver process starts, and does not monitor memory during execution. Runs rejected by the guard are recorded as `BUILD_FAIL` with `peak_rss_mb=None`.

Runtime and memory statistics are computed over `SOLVED_VALID` runs only. Censored timeout measurements are excluded from runtime statistics because timeouts have no finite solve time; they contribute to denominators for solve rates but not to median, mean, or P95 statistics. Predeclared exclusions and live resource-guard rejections are likewise excluded.

All puzzles are generated from base seed 42. One run is performed per instance per configuration; there is no replication across seeds or runs. Consequently, no variance estimates, confidence intervals, or hypothesis tests are available, and all statistics in Section 5 are descriptive single measurements.

Table 2
Aggregate solver-family comparison. Denominators exclude 80 predeclared SAT memory-guard exclusions. Valid rate and timeout rate are computed over attempted runs. Median solve time is over successful runs only.

Metric	SAT (7 configs)	CP-SAT (1 config)	Backtracking (1 config)
Solver configurations	7	1	1
Recorded matrix cells	840	120	120
Predeclared exclusions	80	0	0
Attempted runs	760	120	120
Valid solutions	431	64	40
Valid solve rate (%)	56.71	53.33	33.33
Timeouts	328	56	80
Timeout rate (%) of attempted	43.16	46.67	66.67
Live resource-guard rejections	1	0	0
Invalid solutions	0	0	0
Median solve time, valid (s)	0.009367	0.033353	0.0010

4.4. Environment

Hardware: Fedora Linux 44 (Xfce), Intel Core i5-1135G7 CPU @ 2.40 GHz, 8 GiB installed RAM (7.5 GiB available). No CPU frequency scaling, core count, or memory bandwidth were recorded.

Software: Python 3.14.8 (confirmed by `data/dataset_info.json`). PySAT (backends: cadical153, glucose4, minisat22), OR-Tools CP-SAT, and psutil are used. Gurobi and CPLEX adapters exist in the codebase but were not executed (license restrictions). Exact solver version pins beyond backend names are not recorded, and no environment lock file is present in the repository (see Section 7).

5. Results

5.1. Aggregate solver-family comparison

Table 2 and Figure 2 report the aggregate comparison across the three solver families. Denominators strictly exclude predeclared memory-guard exclusions. Valid-solution rate is Valid Solutions/Attempted Runs. Timeout rate is timeouts divided by attempted runs.

SAT and CP-SAT achieve comparable valid solve rates (56.71% vs. 53.33%), but the aggregation structures differ: SAT pools seven configurations across 760 attempted runs, whereas CP-SAT is a single configuration across 120 runs and backtracking is a single configuration across 120 runs. Pooling seven SAT configurations gives SAT more "at-bats" per instance; per-configuration results are reported in Section 5.2. Backtracking's 33.33% valid rate reflects 40 valid solutions, all at $N = 9$, and 80 timeouts, all at $N = 36$ and $N = 81$.

All 535 valid solutions (431 SAT + 64 CP-SAT + 40 backtracking) passed independent validation; zero invalid solutions were recorded across all families. No solver errors or crashes were recorded in the primary results.

5.2. Per-SAT-configuration breakdown

Table 3 and Figure 3 report results for each individual SAT configuration. Predeclared exclusions and the live resource-guard rejection are reported separately.

The product encoding achieves the most valid solves among SAT configurations (64), matching CP-SAT's total (64). The commander encoding is second (63), with one live resource-guard rejection at $N = 81$. Pairwise and binary show high rates on attempted runs (77.50%, 75.00%) only because all 40 of their $N = 81$ instances are predeclared excluded by the memory guard; their rates over all 120 recorded cells would be 51.67% and 50.00%, respectively. The differences between product (64), commander (63), and sequential/pairwise (62) are small (one to two solves) and no significance test is performed.

CP-SAT (64 valid on 120 attempted, 53.33%) matches product's total but is a single configuration; per-configuration SAT comparisons should not be interpreted as a ranking of encoding quality without replication.

Table 3
Per-SAT-configuration breakdown across all 120 instances. Median solve time is over successful runs only.

Solver	Recorded	Predecl.	Attempted	Valid	Rate (%)	Timeouts	Guard live	Median (s)
sat_pairwise_cadical153	120	40	80	62	77.50	18	0	0.009861
sat_sequential_cadical153	120	0	120	62	51.67	58	0	0.007760
sat_binary_cadical153	120	40	80	60	75.00	20	0	0.011180
sat_commander_cadical153	120	0	120	63	52.50	56	1	0.008905
sat_product_cadical153	120	0	120	64	53.33	56	0	0.009016
sat_sequential_glucose4	120	0	120	60	50.00	60	0	0.007728
sat_sequential_minisat22	120	0	120	60	50.00	60	0	0.006915

Table 4
Results by grid size N and removal ratio r. Each stratum has 20 instances. SAT attempted runs reflect 7 configurations minus predeclared exclusions at $N = 81$. Median solve time is over successful runs only; "—" denotes no valid solves, or a median not computed per stratum.

N	r	Solver	Attempted	Valid	Rate (%)	Timeouts	Guard live	Invalid	Median (s)
9	0.5	SAT	140	140	100.00	0	0	0	0.008018
9	0.5	CP-SAT	20	20	100.00	0	0	0	0.003896
9	0.5	Backtracking	20	20	100.00	0	0	0	—
9	0.9	SAT	140	140	100.00	0	0	0	0.008288
9	0.9	CP-SAT	20	20	100.00	0	0	0	0.033160
9	0.9	Backtracking	20	20	100.00	0	0	0	—
36	0.5	SAT	140	13	9.29	127	0	0	6.981681
36	0.5	CP-SAT	20	4	20.00	16	0	0	7.008288
36	0.5	Backtracking	20	0	0.00	20	0	0	—
36	0.9	SAT	140	138	98.57	2	0	0	1.508942
36	0.9	CP-SAT	20	20	100.00	0	0	0	2.270578
36	0.9	Backtracking	20	0	0.00	20	0	0	—
81	0.5	SAT	100	0	0.00	99	1	0	—
81	0.5	CP-SAT	20	0	0.00	20	0	0	—
81	0.5	Backtracking	20	0	0.00	20	0	0	—
81	0.9	SAT	100	0	0.00	100	0	0	—
81	0.9	CP-SAT	20	0	0.00	20	0	0	—
81	0.9	Backtracking	20	0	0.00	20	0	0	—

5.3. Results by grid size and clue-removal ratio

Figure 4 and Table 4 report results by stratum for SAT (pooled across seven configurations), CP-SAT, and backtracking.

$N = 9$ **(standard Sudoku).** All three families achieve 100% solve rates on both ratios. CP-SAT is faster than SAT at $r = 0.5$ (median 0.0039s vs. 0.0080s); SAT is faster than CP-SAT at $r = 0.9$ (median 0.0083s vs. 0.0332s, approximately 4×). Backtracking solves all 40 $N = 9$ instances with a median solve time of about 1 ms (Table 5). These timing differences are single-median comparisons with no significance test; distributions may overlap.

$N = 36, r = 0.5$ **(hardness cliff).** Both SAT (9.29%) and CP-SAT (20.0%) fall below 21%; backtracking solves 0 of 20 (all timeouts). Median valid solve times cluster tightly around 7.0s for SAT and CP-SAT, near the 10s timeout, indicating that successful solves are themselves slow. This stratum represents the most difficult condition tested.

$N = 36, r = 0.9$ **(underconstrained/easy regime).** CP-SAT achieves 100%; SAT achieves 98.57% (138/140), with 2 timeouts; backtracking solves 0 of 20. SAT's median (1.51s) is faster than CP-SAT's (2.27s).

$N = 81$ **(large grid).** Zero valid solutions for all three families across both ratios. SAT pairwise and binary are predeclared excluded (40 per stratum); the remaining five SAT configurations and CP-SAT were attempted and all timed out (99–100 timeouts per SAT stratum plus 1 live guard rejection at $r = 0.5$; 20 timeouts per CP-SAT stratum; 20 timeouts per backtracking stratum). $N = 81$ is intractable within

Table 5
Wall-clock runtime statistics (seconds) over solved runs only, by solver configuration and grid size N. Timeouts and predeclared exclusions are excluded. "—" denotes no valid solves at that size.

Configuration	$N = 9$ (40 solved)			$N = 36$ (solved count varies)		
	Median	Mean	P95	Median	Mean	P95
sat/pairwise/cadical153	0.0094	0.0093	0.0103	3.3844	3.7409	4.6272
sat/sequential/cadical153	0.0074	0.0073	0.0083	1.1384	2.6829	5.7728
sat/binary/cadical153	0.0107	0.0107	0.0134	2.1846	3.2338	6.7114
sat/commander/cadical153	0.0085	0.0084	0.0095	0.9294	1.5970	3.6838
sat/product/cadical153	0.0086	0.0085	0.0094	1.1835	2.5690	7.8467
sat/sequential/glucose4	0.0072	0.0073	0.0083	2.1766	2.5219	4.5329
sat/sequential/minisat22	0.0066	0.0066	0.0075	0.9314	0.9646	1.2381
CP-SAT	0.0161	0.0185	0.0351	2.3890	2.9944	6.9496
Backtracking	0.0010	0.0010	0.0012	—	—	—

Table 6
Peak memory usage (MiB) over solved runs only, by solver configuration and grid size N. Count is the number of solved runs. "—" denotes no valid solves or no memory data.

Configuration	$N = 9$			$N = 36$		
	Median	Max	Count	Median	Max	Count
sat/pairwise/cadical153	102.31	103.00	40	914.97	983.71	22
sat/sequential/cadical153	101.95	102.15	40	288.43	314.17	22
sat/binary/cadical153	102.54	103.10	40	414.62	418.95	20
sat/commander/cadical153	101.68	101.87	40	283.91	292.39	23
sat/product/cadical153	101.93	103.14	40	257.17	266.28	24
sat/sequential/glucose4	101.42	101.61	40	249.12	251.25	20
sat/sequential/minisat22	101.09	101.94	40	227.91	228.49	20
CP-SAT	107.05	107.91	40	174.94	176.62	24
Backtracking	—	—	—	—	—	—

the 10-second timeout for all evaluated approaches; this is a timeout-bound conclusion only, not a claim of unsolvability with larger budgets.

5.4. Runtime and memory usage

Table 5 reports wall-clock runtime statistics over solved (`SOLVED_VALID`) runs only, grouped by solver configuration and grid size N. Timeout-censored runs and predeclared memory-guard exclusions are excluded. At $N = 81$, no solver family produced a valid solution within the 10.0-second timeout, so no runtime statistics are reported for that size. Backtracking solves zero instances at $N = 36$ and $N = 81$ (all timeouts), so no runtime statistics exist for those strata.

Table 6 reports peak memory usage (peak RSS above baseline) over solved runs only, in mebibytes (MiB). The backtracking results file does not record peak RSS, so no memory statistics are available for backtracking.

Several patterns emerge. First, all SAT configurations and CP-SAT consume roughly 101–108 MiB at $N = 9$, reflecting a common Python/PySAT or OR-Tools baseline plus a small CNF. At $N = 36$, memory usage diverges substantially by encoding: pairwise (the largest CNF, approximately 85 million clauses at $N = 81$) peaks at 915 MiB at $N = 36$, while MiniSat 2.2 with sequential encoding peaks at only 228 MiB. CP-SAT's memory (175–177 MiB at $N = 36$) is lower than most SAT configurations despite its integer-variable model, consistent with avoiding CNF materialization.

Second, runtime at $N = 9$ is uniformly fast (under 0.02 seconds median) for all families. At $N = 36$, MiniSat 2.2 with sequential encoding is fastest (0.93 s median), while pairwise is slowest (3.38 s median). CP-SAT's median at $N = 36$ (2.39 s) is competitive with several SAT configurations. Backtracking solves

all $N = 9$ instances in about 1 ms (median 1.0 ms, P95 1.2 ms) but solves zero instances at $N = 36$ and $N = 81$ (all timeouts).

Third, median runtime at $N = 36$ ranges from 0.93 s to 3.38 s across configurations, while P95 values reach up to 7.85 s (product encoding) and exceed 6 s for several others, approaching the 10-second timeout boundary. Even successful solves at $N = 36$ are therefore slow relative to $N = 9$.

Finally, no configuration solves any $N = 81$ instance within the 10-second timeout. Pairwise and binary SAT encodings at $N = 81$ are predeclared excluded by the memory guard (80 runs total); the remaining five SAT configurations, CP-SAT, and backtracking were all attempted and all timed out.

5.5. Encoding size versus runtime

Per-run `stats` fields record primary variable count (N^3), auxiliary variable count (encoding-dependent), total variable count, clause count, and clue count for every SAT run. These data exist in the raw CSV but have not been aggregated or correlated with runtime in this study. The relationship between encoding size and runtime is therefore stated as unanalyzed; no correlation, regression, or significance claim is made. For reference, at $N = 81$: primary variables $N^3 = 531{,}441$; exactly-one groups $4N^2 = 26{,}244$; pairwise AMO per group $N(N-1)/2 = 3{,}240$ clauses (total pairwise AMO clauses ≈ 85 million, motivating predeclared exclusion); sequential counter $N - 1 = 80$ auxiliary variables per group; binary $\lceil \log_2 81 \rceil = 7$ auxiliary variables per group. Figure 5 shows clause counts by encoding together with observed outcome counts.

6. Discussion

6.1. Interpretation of aggregate results

SAT's 56.71% valid rate (431/760) and CP-SAT's 53.33% (64/120) are numerically close, but the comparison is not symmetric. SAT's denominator aggregates seven configurations, so each instance contributes up to seven attempted runs; CP-SAT contributes one. Consequently, SAT's absolute solve count (431) is much larger than CP-SAT's (64) even though the per-attempted-run rates are similar. Per-configuration SAT results range from 50.00% (sequential with Glucose 4 and MiniSat 2.2) to 77.50% on attempted runs (pairwise, but with 40 $N = 81$ exclusions). The product encoding's 64 valid solves matches CP-SAT's total, but the one-solve margin over commander (63) is too small to support a ranking claim without replication.

Backtracking's 33.33% valid rate (40/120) is entirely attributable to $N = 9$, where it solves all 40 instances in about 1 ms median. Its 0% at $N = 36$ and $N = 81$ (80 timeouts) reflects this specific implementation's scaling limits on the tested instances, not a general property of classical backtracking. The implementation uses MRV + forward checking without AC-3, LCV, or degree heuristics; stronger configurations were not evaluated.

6.2. Algorithmic scaling

At $N = 9$, all three families solve 100%. At $N = 36, r = 0.9$, SAT and CP-SAT remain effective while backtracking times out on all 20 instances. At $N = 36, r = 0.5$, all families struggle (below 21%). At $N = 81$, no family solves any instance within 10s. The empirical difficulty ordering is $N = 9 \ll N = 36, r = 0.9 < N = 36, r = 0.5 \ll N = 81$, but with only 20 instances per stratum and a single seed, this ordering is descriptive, not generalizable.

The stark contrast between $r = 0.9$ (easy) and $r = 0.5$ (hard) at $N = 36$ is consistent with the intuition that underconstrained puzzles are easier for propagation-driven solvers, while highly constrained puzzles with few clues create large search spaces. However, removal ratio is an experimental condition, not a difficulty label, and the generator documentation explicitly cautions against interpreting r as difficulty.

6.3. Representation effects

SAT's one-hot Boolean encoding with AMO constraints produces large CNFs (hundreds of thousands of variables and millions of clauses at $N = 81$), which motivated predeclared exclusion of pairwise and binary encodings at $N = 81$. CP-SAT's compact integer-variable model with all-different constraints avoids CNF blow-up but still times out at $N = 81$ within 10s, indicating that the difficulty at $N = 81$ is not solely a formula-size phenomenon. Backtracking's domain-based search scales poorly beyond $N = 9$ on this

implementation, consistent with the exponential worst case of backtracking, but the measured timeouts reflect the specific heuristic configuration.

6.4. Search heuristics

Backtracking's MRV + forward checking is effective at $N = 9$ (solves in about 1 ms) but insufficient at $N = 36$ and $N = 81$ under the 10s budget. The absence of LCV, degree heuristics, and AC-3 (all confirmed absent from the implementation) may contribute to this scaling limit, but no ablation was performed to isolate their effects. SAT CDCL backends differ (CaDiCaL, Glucose, MiniSat) but their per-configuration results are clustered (50–52% valid rates for sequential across backends), suggesting backend choice is secondary to encoding and instance difficulty within the tested range.

6.5. Limits of cross-family comparison

Comparisons across solver families are limited by (i) different aggregation structures (7 vs. 1 vs. 1 configurations), (ii) predeclared exclusions affecting only SAT pairwise/binary at $N = 81$, (iii) timeout censoring (timeouts have no finite solve time), and (iv) survivor bias in median solve times (computed over successful runs only, excluding censored timeouts). These limitations are structural to the benchmark design and are disclosed throughout.

6.6. What is not claimed

We do not claim: (i) universal superiority of SAT over CP-SAT or backtracking, or vice versa; (ii) that the product encoding is provably the best SAT encoding (its advantage over commander is one solve, 64 vs. 63); (iii) statistical significance for any observed difference; (iv) that $N = 81$ Sudoku is unsolvable by any approach (only unsolvable within 10s); (v) a phase transition (the observed difficulty differences lack theoretical or statistical support); or (vi) that classical backtracking in general cannot solve larger Sudoku instances (the timeouts reflect this specific implementation, configured with MRV + forward checking and no AC-3, on the tested instances).

7. Threats to Validity

1. **Single seed and single automorphism orbit.** All puzzles are generated from base seed 42 and sampled from the automorphism orbit of one base pattern. Results may not generalize to other seeds or grid distributions.
2. **Small sample size.** Twenty puzzles per stratum (120 total) limits the precision of solve-rate estimates; no confidence intervals are computed.
3. **No replication.** One run per instance per configuration; no variance or reproducibility estimates are available.
4. **No confidence intervals or hypothesis tests.** All statistics are descriptive single measurements; no inferential claims are made.
5. **Incomplete environment information.** Exact solver versions beyond backend names are not pinned, and no environment lock file is present (`environment_lock_present=false`). Only the operating system, CPU, RAM, and Python version are recorded (Section 4).
6. **Predeclared memory exclusions.** Eighty SAT runs (pairwise and binary at $N = 81$) are predeclared excluded by the 60% RAM guard; their performance at $N = 81$ is unknown. This creates an aggregation asymmetry favoring pairwise/binary rates on attempted runs.
7. **Timeout censoring and survivor bias.** The 10.0s timeout censors runtimes; medians are computed over successful runs only and exclude censored timeouts, which cluster near the timeout at $N = 36, r = 0.5$.
8. **Backtracking results are implementation-specific.** Backtracking's 0% at $N = 36$ and $N = 81$ reflects this implementation (MRV + forward checking, no AC-3/LCV/degree), not classical backtracking generally.
9. **Three pre-existing failing tests.** `test_full_v1.py` reports three failing tests (the file documents test expectations around matrix shape, resume behavior, and solution-payload preservation). Their impact on result validity has not been established; they are not assumed irrelevant.

Table 7
On-disk storage footprint of benchmark artifacts.

Artifact	Size	Notes
Puzzle dataset (6 JSONL files, 120 puzzles)	1.66 MiB	`data/puzzles/`
Dataset manifest	21.0 KiB	120 rows
SAT + CP-SAT results	351.5 KiB	960 runs
Backtracking results	11.3 KiB	120 runs
Source code (`src/`)	536 KiB	Solvers, generator, validator, runner
Total (puzzles + results + source)	2.2 MiB	Approximate

10. **Two removal ratios only.** $r \in \{0.5, 0.9\}$; no data for intermediate ratios.
11. **Non-unique solutions.** Puzzles may admit multiple valid solutions; correctness is any valid completion matching clues.
12. **No commercial solvers.** Gurobi and CPLEX are excluded by license restrictions; their performance is unknown.
13. **Limits on external generalization.** Findings apply to the tested dataset, solver versions, timeout, and memory guard only; they do not generalize to other Sudoku distributions, solver configurations, or time budgets.

8. Conclusion

This paper presents an empirical benchmark of SAT (seven configurations, five AMO encodings, three CDCL backends), OR-Tools CP-SAT, and classical backtracking on 120 generalized Sudoku instances ($N \in \{9, 36, 81\}$; $r \in \{0.5, 0.9\}$; 20 per stratum) with independent solution validation. Across 1,000 attempted runs, SAT achieves 56.71% (431/760), CP-SAT achieves 53.33% (64/120), and backtracking achieves 33.33% (40/120), all with zero invalid solutions. Both SAT and CP-SAT solve 100% of $N = 9$ instances; all families fall below 21% at $N = 36, r = 0.5$; and no family solves any $N = 81$ instance within the 10-second timeout. The product encoding records the most valid solves among SAT configurations (64), but the margin over commander (63) is one solve and unsupported by significance testing.

The benchmark contributes a documented, reproducible dataset and raw results for replication, explicit accounting of predeclared exclusions and timeout censoring, and descriptive evidence that SAT and CP-SAT are comparable on this benchmark while backtracking's results are confined to $N = 9$ for this implementation.

What remains uncertain: the performance of pairwise and binary SAT encodings at $N = 81$ (predeclared excluded); whether encoding size correlates with runtime (unanalyzed); whether results replicate across seeds or runs; and the exact hardware/software environment. Future studies that would most improve confidence: replication across multiple seeds and puzzle distributions; confidence intervals and hypothesis tests; environment locking; evaluation of backtracking with AC-3, LCV, and degree heuristics; larger time budgets at $N = 81$; and inclusion of the unexecuted $N = 81$ pairwise/binary SAT configurations with sufficient memory.

This is a descriptive, hypothesis-generating benchmark, not a confirmatory study.

Data availability

The source code and benchmark artifacts are available at GitHub repository: Sudoku-SAT.

Dataset: `data/manifest.csv` and `data/puzzles/`. Raw results: `data/results/benchmark_sat_vs_nonsat.csv` (primary, 960 runs) and `data/results/backtracking_results.csv` (backtracking, 120 runs). Dataset metadata: `data/dataset_info.json`. Table 7 reports the on-disk size of these artifacts; these are storage measurements, distinct from peak memory (RSS) during execution.

No solver binaries or shared libraries are measured separately: the benchmark invokes solvers through Python bindings (PySAT, OR-Tools) within a single Python process, so executable and library sizes are subsumed by the Python interpreter and its shared dependencies.

References

, 2007. An interactive constraint-based approach to sudoku, in: Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence (AAAI-07). Verified: PDF available at https://cdn.aaai.org/AAAI/2007/AAAI07-362.pdf; constraint-based Sudoku solver, references Sudoku NP-completeness.

Chen, J., 2010. A new sat encoding of the at-most-one constraint. PARTIALLY VERIFIED: referenced as [4] in [hn2013efficient]; source of product AMO encoding; full bibliographic details not confirmed against a primary source.

Frisch, A.M., Giannaros, P.A., 2010. Sat encodings of the at-most-k constraint: some old, some new, some fast, some slow. PARTIALLY VERIFIED: referenced in multiple sources; source of binary at-most-one encoding; full bibliographic details not confirmed against a primary source.

Google, 2026. Google OR-Tools. Verified: GitHub repository at https://github.com/google/or-tools; CP-SAT solver documented as hybrid constraint programming + CDCL SAT engine.

Hn, e.a., 2013. An Efficient Encoding of the at-most-one Constraint. Technical Report. TU Dresden. Verified: technical report available at https://iccl.inf.tu-dresden.de/w/images/d/d7/Hn-tr-2013.pdf; survey of AMO encodings with clause/variable count comparisons.

Klieber, W., Kwon, G., 2007. Effective use of microsoft solver foundation in constraint-based software verification. PARTIALLY VERIFIED: referenced as [2] in [hn2013efficient]; source of commander AMO encoding; full bibliographic details not confirmed against a primary source.

Kwon, G., 2016. Optimized cnf encoding for sudoku puzzles. PARTIALLY VERIFIED: PDF accessible at https://www.cs.cmu.edu/ hjain/papers/sudoku-as-SAT.pdf; venue, year, and DOI not confirmed against a primary source; cited for context only.

Lynce, I., Ouaknine, J., 2006. Sudoku as a SAT problem. AI*MATH Verified: PDF available at https://sat.inesc-id.pt/ ines/publications/aimath06.pdf; introduces one-hot Boolean SAT encoding for Sudoku with exactly-one constraints per cell, row, column, box.

Nguyen, V.H., 2020. Empirical study on SAT-encodings of the at-most-one constraint, in: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing (SAC 2020). Verified: ACM Digital Library, DOI 10.1145/3426020.3426170; empirical comparison of pairwise, sequential, binary, commander, product, bimander encodings across three SAT solvers.

Sheng, A., Reeves, J.E., Heule, M.J.H., 2025. Reencoding unique literal clauses. PARTIALLY VERIFIED: associated with SAT 2025 (LIPIcs.SAT.2025.29); discusses sequential counter vs order encoding for exactly-one constraints; full bibliographic details not confirmed against a primary source.

Simonis, H., 2005. Sudoku as a constraint problem, in: Working notes of the CP 2005 Workshop on Modeling and Reformulating Constraint Satisfaction Problems. Verified via reference in [aaai2007interactive]; models Sudoku with all-different constraints on rows, columns, boxes.

Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. Verified: sequential counter encoding for cardinality constraints; O(n) auxiliary variables and O(n) clauses for at-most-one.

Yato, T., 2003. Sudoku is np-complete. Verified via reference in [aaai2007interactive]; reduction from Latin Square completion establishing NP-completeness of Sudoku.

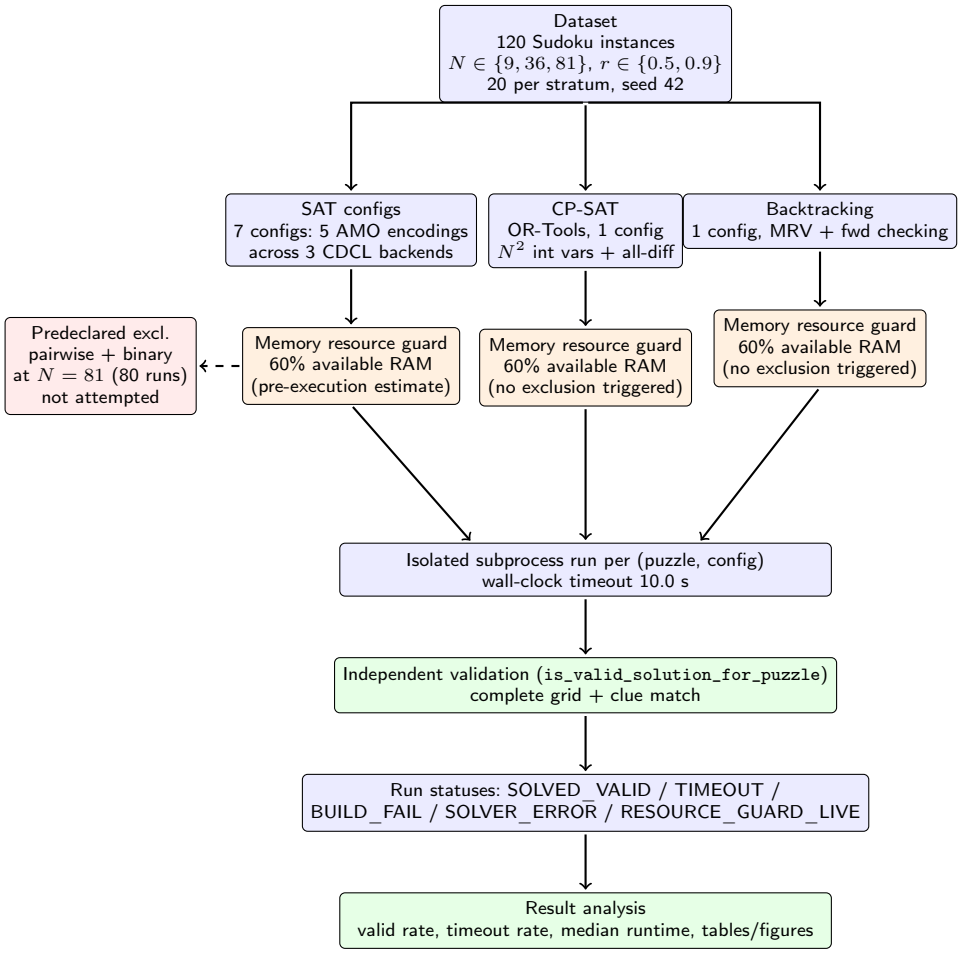


Figure 1: Experimental workflow. Twenty puzzles per stratum are paired with solver configurations (seven SAT, one CP-SAT, one backtracking). Each run executes under a 10.0-second wall-clock timeout and a 60% available-memory resource guard; pairwise and binary SAT encodings at $N = 81$ are predeclared excluded by the guard. Every returned solution is independently verified by `is_valid_solution_for_puzzle` before a run is classified as `SOLVED_VALID`. Result statuses feed the aggregate, per-configuration, and stratified analyses in Section 5. Excluded runs are not attempted and therefore do not enter the run or validation stages.

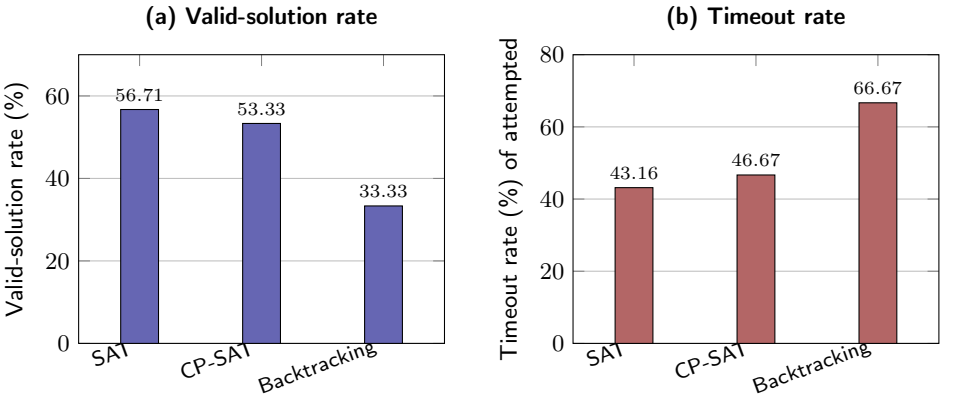


Figure 2: Overall solver-family comparison. Panel (a): valid-solution rate (valid solutions / attempted runs). Panel (b): timeout rate (timeouts / attempted runs). SAT pools seven configurations across 760 attempted runs; CP-SAT and backtracking are single configurations across 120 attempted runs each. SAT's denominator excludes 80 predeclared memory-guard exclusions (pairwise and binary at $N = 81$). All rates are descriptive single measurements with no significance testing.

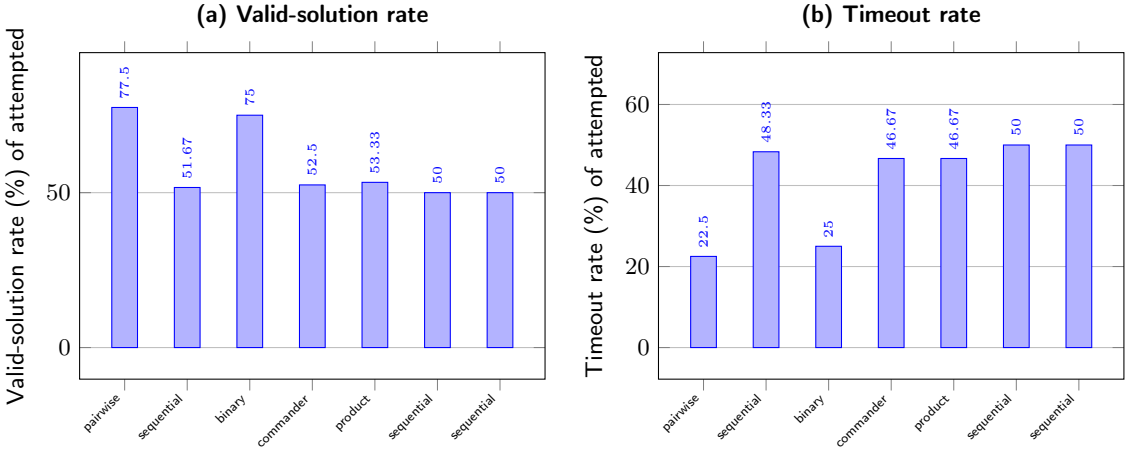
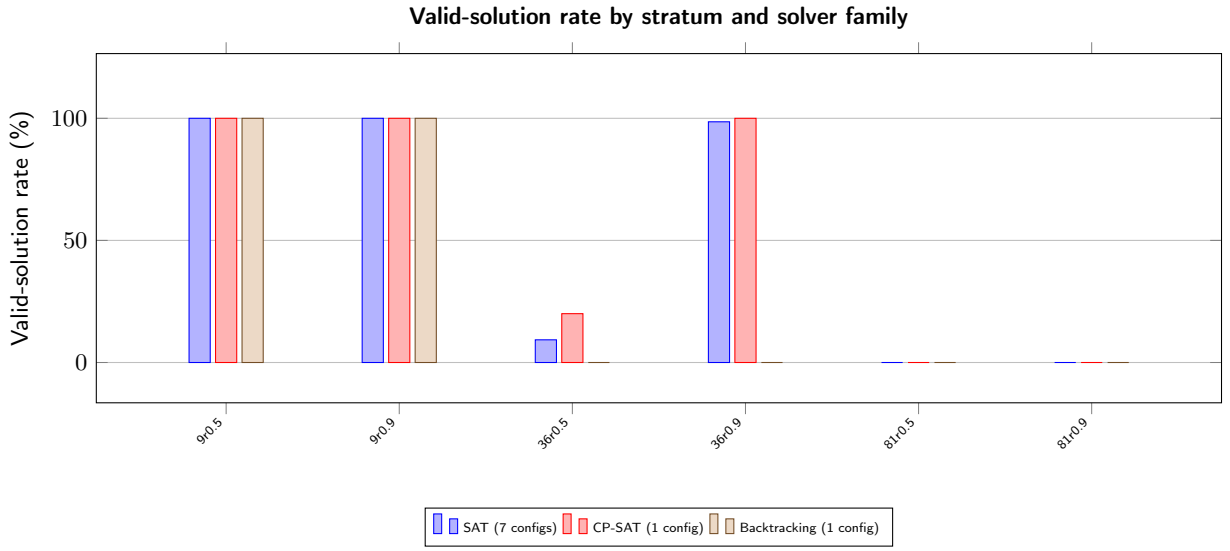
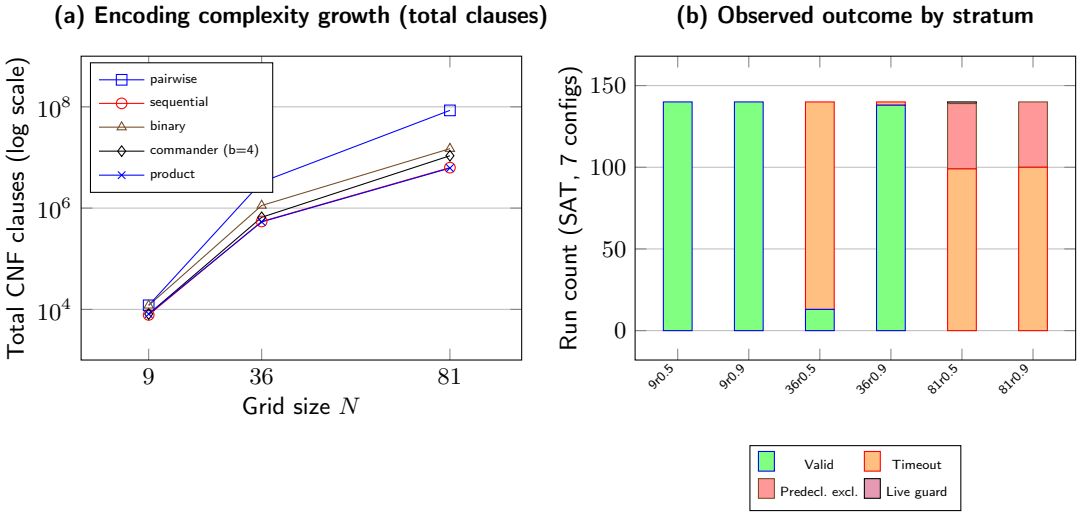


Figure 3: Per-SAT-configuration comparison. Panel (a): valid-solution rate of attempted runs. Panel (b): timeout rate of attempted runs. Pairwise and binary each show inflated rates on attempted runs because 40 of their $N = 81$ instances are predeclared excluded by the memory guard (attempted = 80 rather than 120); over all 120 recorded cells their rates would be approximately 51.67% and 50.00%. Commander has one live resource-guard rejection. The spread among configurations is small (50.00%–77.50% valid on attempted runs) and no significance test is performed.



Attempted runs per stratum: SAT [140,140,140,140,100,100] (40 predeclared exclusions removed at each N=81 stratum); CP-SAT [20,20,20,20,20,20]; Backtracking [20,20,20,20,20,20]. Zero-height bars denote 0% valid rate; no valid-solution median exists at N=81 or for backtracking at N=36/N=81.

Figure 4: Stratified performance by grid size N and removal ratio r. Valid-solution rate per solver family across the six strata (20 instances per stratum). SAT pools seven configurations; its attempted count per stratum is 140 at $N = 9$ and $N = 36$, and 100 at $N = 81$ after removing 40 predeclared memory-guard exclusions (pairwise and binary). CP-SAT and backtracking attempt 20 runs per stratum. Zero-height bars denote 0% valid rate; excluded runs are not drawn as bars but are noted in the caption. SAT's valid rate at $N = 81$ is 0% (0 valid / 100 attempted); CP-SAT and backtracking also reach 0% at $N = 81$. The $N = 36, r = 0.5$ stratum is the hardest for all families (below 21%).



Clause counts exclude per-stratum clue-unit variation (shown values use representative clue counts; exact per-run values are in the `stats.clauses` field of `data/results/benchmark_sat_vs_nonsat.csv`). Pairwise and binary at $N = 81$ exceed the 60% available-RAM guard and were predeclared excluded (no point plotted). Panel B run counts sum to recorded cells per stratum (140 at $N = 9/N = 36$; 100 attempted + 40 excluded at $N = 81$). Runtime medians are over successful runs only and are timeout-censored; no correlation between clause count and runtime has been statistically analyzed.

Figure 5: Encoding complexity versus observed outcome (descriptive; no correlation claimed). Panel (a): total CNF clauses versus grid size N for each AMO encoding, computed exactly from the formulas in `src/sudoku/solvers/sat.py` (one ALO clause plus AMO clauses per exactly-one group, plus clue-unit clauses). Pairwise and binary at $N = 81$ exceed the 60% available-RAM memory guard and were predeclared excluded (no data point plotted). Panel (b): observed outcome counts per stratum for SAT (pooled seven configurations), showing valid, timeout, predeclared-excluded, and live-guard-rejection runs. Runtimes are successful-run-only and timeout-censored; no statistical correlation between clause count and runtime has been analyzed. Pairwise/binary at $N = 81$ are excluded rather than timed out, so their runtime performance at that size remains unknown.